Utilizando Markedown junto com a extensão do VS Code Markdown Preview Enhanced para criar anotações e referências

William de Paula

Introdução

Algumas dicas e exemplos de uso do Markdown junto com a extensão Markdown Preview Enhanced do VS Code.

- Introdução
- Instalação
- Utilizando o Markdown
  - Marcadores do markdown
- Vídeos do Youtube
- Funcionalidades do Markdown Preview Enhanced
  - Criando sumário automaticamente
  - Conversão ao salvar
  - Importando arquivos externos
    - Importando imagens
    - Importando arquivos CSV
    - Importando código fonte
  - Latex
  - Notas de rodapé
  - Blocos de código
    - Executando código javascript
    - Executando código Python
  - Diagramas
    - Flow Charts
    - Diagrama de sequencia
    - Diagrams com GraphViz
      - Gráficos não direcionados
  - Slides

Instalação

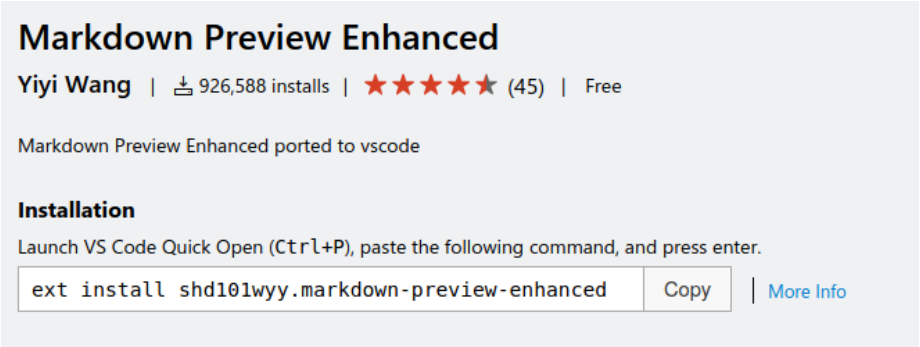
Baixe e instale o VS Code no seu computador, caso não tenha ainda.

Para instalar a extensão Markdown Preview Enhanced, com o VS Code aberto você pode ir até a lateral e clicar no painel de extensões

Pesquise Markdown Preview Enhanced na caixa de pesquisa

Clique no botão `Install`, pronto!

Outra forma mais rápida é acessando o site da extensão Markdown Preview Enhanced e copiar o código de instalação



Com o código copiado, volte para o VS Code, abra o lançador de comandos `(CTRL+P)` ou `F1` cole o comando e aperte `ENTER`

Com a extensão instalada, basta abrir ou criar um arquivo com a extensão `.md` que o VS Code irá detectar e habilitar a opção de abrir o Preview do que está sendo editado, clique no botão no canto superior direito

para ativar.

Utilizando o Markdown

Markdown é uma linguagem de marcação de texto fácil de aprender e usar, inicialmente ele foi criado para ser um marcador de texto mais fácil e menos verboso de usar para criar textos HTML, hoje ele pode ser facilmente convertido para vários formatos de documentos diferentes:

- HTML
- PDF
- World

- ODT
- RTF

e por ter uma licença aberta, vez ou outra surge algum outro formato de exportação.

Textos em markdown são documentos de texto puro, que podem ser abertos em qualquer computador, celular, tablet, notebook, etc, e que utilizam para estilização caracteres não-alfabéticos, como #, * e que serão convertidos para títulos, listas, itálicos, negrito, imagens, etc.

Por exemplo para marcar uma frase com negrito, escreva ela dentro dos marcadores ** : `**frase em negrito**` tem como resultado **frase em negrito**

Marcadores do markdown

Abaixo estão alguns marcadores:

```
Texto com ênfase:

*enfatizado* (e.g., itálico)

**fortemente enfatizado** (e.g., negrito)

Código:

`código`

Listas:

* Um item em uma lista não ordenada
* Outro item em uma lista não ordenada

1. Um item em uma lista ordenada
2. Outro item em uma lista ordenada

Cabeçalhos:

# Cabeçalho de primeiro nível

#### Cabeçalho de quarto nível

Sintaxe alternativa para os dois primeiros cabeçalhos:

Cabeçalho de primeiro nível
====================

Cabeçalho de segundo nível
```

```
--------------------

Citações:

> Esse texto será envolto pelo elemento HTML blockquote.

Links:

[Texto do link](http://example.com/ "Propriedade title, opcional")

Imagens:

![Texto da propriedade alt](/pasta/da/img.jpg "Propriedade title, opcional")

Lista de Tarefas
- [x] @mentions, #refs, [links](), **formatting**, and <del>tags</del> supported
- [x] list syntax required (any unordered or ordered list supported)
- [x] this is a complete item
- [ ] this is an incomplete item
```

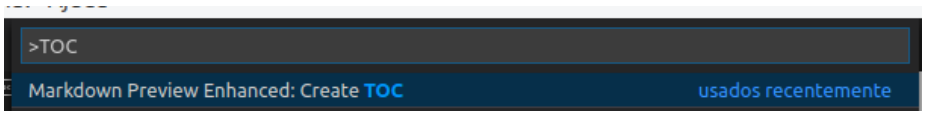
Vídeos do Youtube

Funcionalidades do Markdown Preview Enhanced

As dicas abaixo funcionam com o MPE (Markdown Preview Enhanced) junto com o VS Code. Em outros interpretadores de Markdown eles podem simplesmente não funcionar ou depender de instalação de plugins para funcionar corretamente.

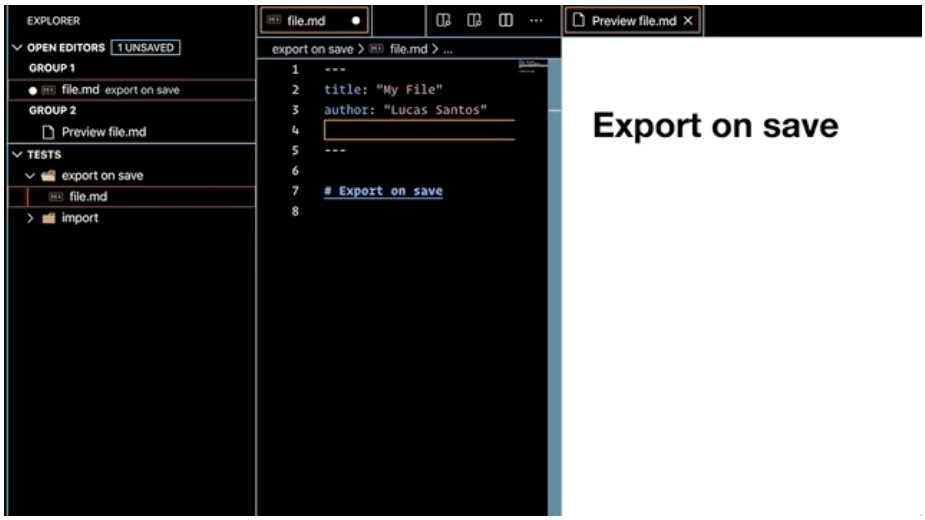
Criando sumário automaticamente

Para criar um súmario do seu texto utilize o comando `CTRL\CMD + SHIFT + P` e digite `TOC`.



O Markdown Preview Enhanced vai buscar todos os títulos com `#` até nível 6 (configurável) e criar um sumário. > É necessário salvar o arquivo para que o sumário seja criado ou atualizado.

Conversão ao salvar



Podemos configurar nosso arquivo markdown para, utilizando o Pandoc salve automaticamente nosso texto em outro formato. Para configurar é necessário adicionar um Front Matter no inicio, antes de qualquer coisa, do nosso arquivo.

```
---
title: "Titulo do nosso arquivo"
author: "Nome do autor"
export_on_save:
  pandoc: true
output:
  pdf_document:
```

```
    path: ./file/path.pdf
---
```

Caso a opção `path:` seja omitida, o arquivo será criado no mesmo diretório do texto.

Importando arquivos externos

Para importar arquivo externos utilize `@import "nome do arquivo"`.

Os formatos suportados são:

- Imagens (jpg, png, gif, apng, svg e bmp)
- CSV (vai ser convertido em uma tabela em markdown)
- .mermaid
- .plantuml ou puml
- .html
- .js (vai ser incluído em um bloco <script>)
- .less ou .css (será incluído como style na página)
- .pdf
- .md

Exemplos:

Importando imagens

```
@import "2020-05-01-09-19-01.png"
```

```
@import "2020-05-01-09-19-01.png"" {width="300px" height="200px" alt="my image"}
```

Importando arquivos CSV

```
@import "planilha.csv"
```

Coluna 1	Coluna 2	Coluna 3	Coluna 4	Coluna 5
Item 1	Item 2	Item 3	Item 4	Item 5
Item 1	Item 2	Item 3	Item 4	Item 5

Importando código fonte

```
{
    "glossary": {
```

```
        "title": "example glossary",
        "GlossDiv": {
            "title": "S",
            "GlossList": {
                "GlossEntry": {
                    "ID": "SGML",
                    "SortAs": "SGML",
                    "GlossTerm": "Standard Generalized Markup Language",
                    "Acronym": "SGML",
                    "Abbrev": "ISO 8879:1986",
                    "GlossDef": {
                        "para": "A meta-markup language, used to create markup languages suc
                        "GlossSeeAlso": ["GML", "XML"]
                    },
                    "GlossSee": "markup"
                }
            }
        }
    }
}
```


Latex

Para utilizando o Latex basta escrever a formula dentro do bloco `$$`:

Exemplo `$$\Delta = b² - 4*ac$$` gera a formula:

$$\Delta = b\check{s} - 4 * ac$$

E a formula `$$\lim_{x\to\infty} \frac{1}{x}$$` tem como resultado:

$$\lim_{x\to\infty} \frac{1}{x}$$

Outro exemplo `$$\sum_{n=1}^{100} n$`

$$\sum_{n=1}^{100} n$$

Notas de rodapé

Também é possível criar notas de rodapé, o código abaixo:

```
William de Paula [^1]

[^1]:[Official WebSite](williamdepaula.github.io/)
```

Gera como resultado

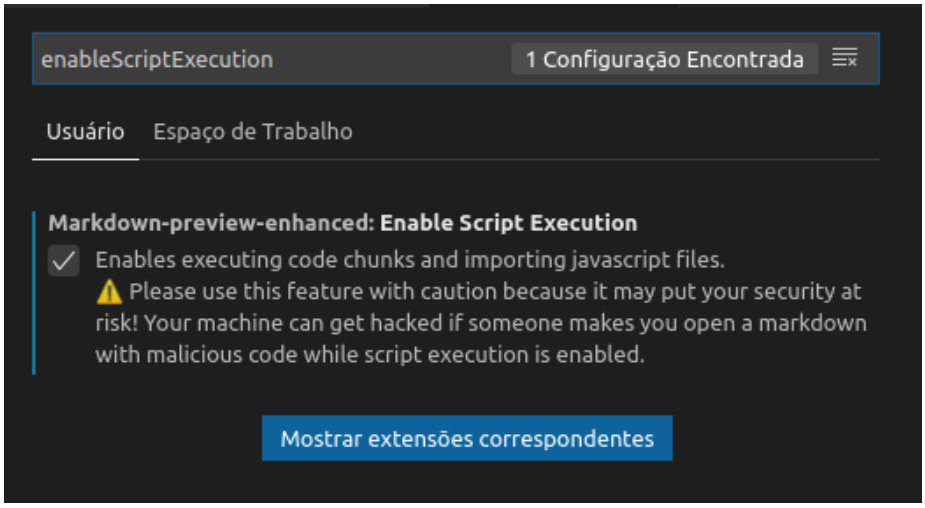
William de Paula [1]

Blocos de código

É possível gerar blocos de código executável dentro do nosso arquivo markdown, código de qualquer tipo pode ser executado, bastando para isso ter o executável na sua máquina.

Por padrão a execução de scripts vem desativada dentro do VS Code, para isso vamos ter que ativa-la dentro das preferencias.

Vá em `Arquivo->Preferências->Configurações` ou use o atalho `CTRL+,` no campo de busca escreva `enableScriptExecution` marque o checkbox para ativar.



Executando código javascript

Abra o bloco com ```javascript {cmd="node"} coloque seu comandos e depois feche com ``` Exemplo:

```javascript
const data = new Date()
console.log(data)
```

Lembrando que o **node** tem que estar instalado na maquina para funcionar corretamente.

Executando código Python

```python
print("Olá mundo!")
```

[1] Site Pessoal

Para este exemplo funcionar o interpretador python deve estar instalado

Também é possível utilizar a biblioteca matplotlib para gerar gráficos

```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4])
plt.show()
```

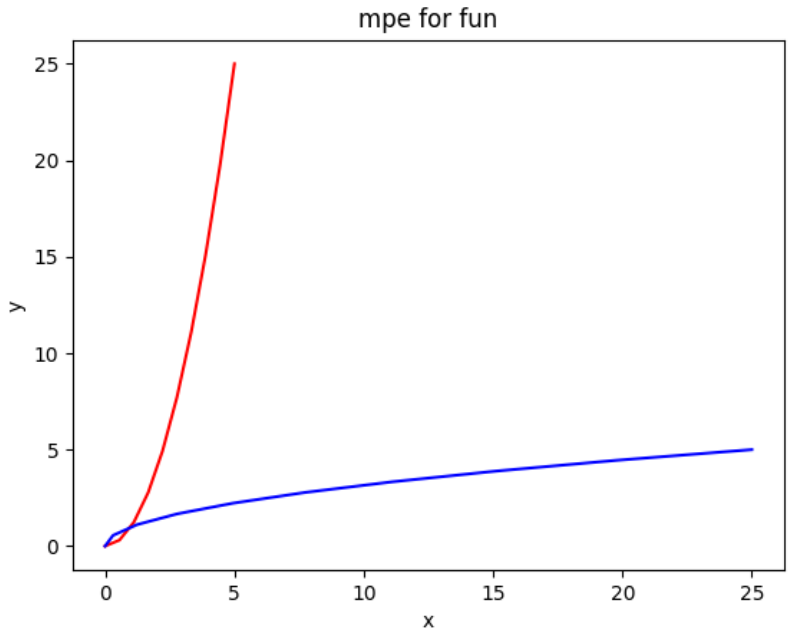
Caso tenha problemas com a importação da biblioteca matplotlib instale, em sistemas Debian like, com o comando abaixo:

```
$ python -m pip install -U matplotlib
```

> O gráfico gerado pode não ser exibido corretamente na exportação com o pandoc, uma forma de contornar isso é salvando a imagem do gráfico e depois importando ele dentro do markdown.

```
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 5, 10)
y = x ** 2

plt.plot(x, y, 'r')
plt.plot(y, x, 'b')
plt.xlabel('x')
plt.ylabel('y')
plt.title('mpe for fun')
plt.savefig('grafico.png')
```



Lembre-se de omitir a linha `plt.show()` para o gráfico não ficar duplicado na apresentação do seu texto

Diagramas

MPE também tem suporte a criação de vários tipos de diagramas e grafos. Ele tem suporte aos seguintes formatos:

- Fluxogramas (flow charts)
- Diagramas de sequencia (sequence diagrams)
- Mermaid
- PantUML
- WaveDrom
- GraphViz
- Vega & Vega-lite
- Ditaa diagrams

Flow Charts

- Crie um bloco de código com a notação ```flowe ele será renderizado utilizando o flowchart.js

```
st=>start: Start:>http://www.google.com[blank]
e=>end:>http://www.google.com
op1=>operation: My Operation
```

```
sub1=>subroutine: My Subroutine
cond=>condition: Yes
or No?:>http://www.google.com
io=>inputoutput: catch something...
para=>parallel: parallel tasks

st->op1->cond
cond(yes)->io->e
cond(no)->para
para(path1, bottom)->sub1(right)->op1
para(path2, top)->op1
```

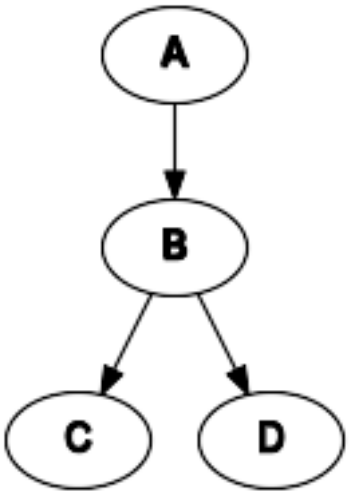
Diagrama de sequencia

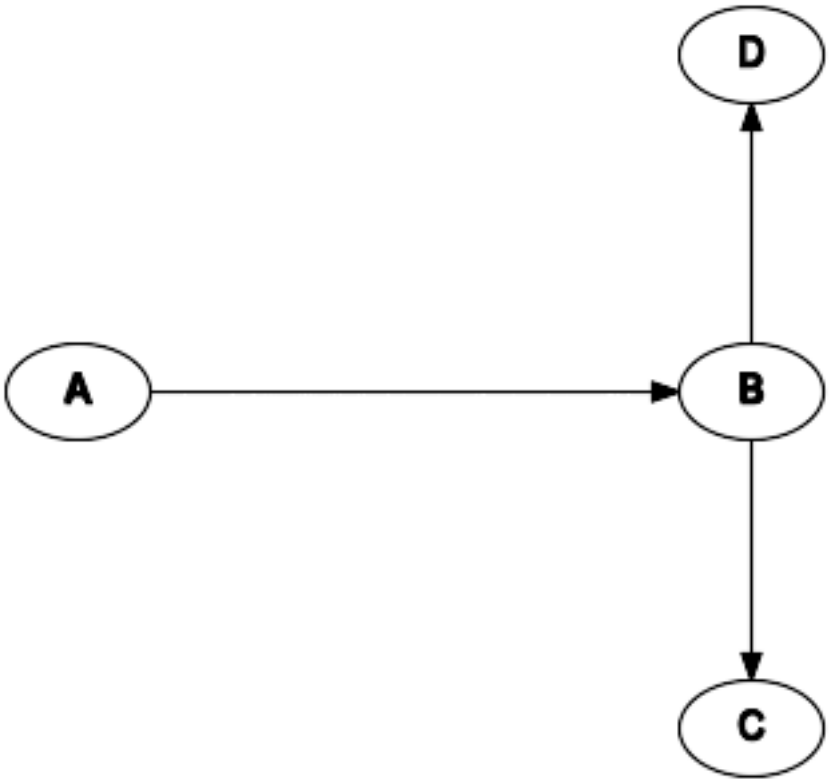
- Crie um bloco de código com a notação ```sequence que ele será renderizado utilizando o js-sequence-diagrams
- Dois temas são suportados: simples(thema padrão) e hand

```
Andrew->China: Says Hello
Note right of China: China thinks\nabout it
China-->Andrew: How are you?
Andrew->>China: I am good thanks!
```

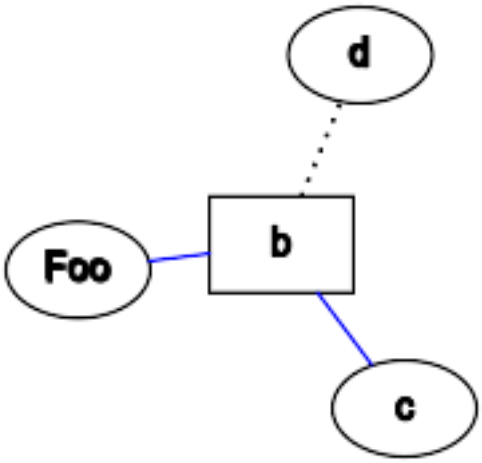
Diagrams com GraphViz

- Crie um bloco de código com a notação ```dot ou ```viz que ele será renderizado utilizando a biblioteca Viz.js.
- É possível escolher uma engine diferente utilizando o marcador {engine="..."}. As engine circo, dot(engine default), neato, osage, ou twopi são suportadas.
- Utiliza a DOT (graph description language) para criar os gráficos.
- Veja a documentação para criar gráficos





Gráficos não direcionados



Slides

É possível criar Slides e apresentações utilizando o MPE junto com markdown. Criei uma apresentação de slides para mostrar como fazer apresentações utilizando markdown.